

Business Health Agent

A hands-on AI engineering project demonstrating LangChain, LangSmith, and Azure-ready agent development for small business financial analysis.

Project Overview

The Business Health Agent is a multi-agent AI system that answers natural language questions about a small business's financial health. A business owner can ask questions like *"Is the business profitable this month?"*, *"Who are my top customers?"*, or *"Am I at risk of a cash flow problem?"* — and receive clear, data-driven answers in plain English.

The system is built entirely in Python, runs in the terminal or Jupyter, and is designed to be deployable on Azure. It reads from a real financial dataset (Kaggle Small Business Financial Dataset 2022–2023) with automatic fallback to mock data, making it self-contained for demonstration purposes.

This project was built as a working, runnable implementation — not a theoretical exercise. Every error encountered during development is documented, every fix is real, and every design decision reflects the kind of judgment expected of a senior AI/Cloud engineer.

Aim and Objectives

Aim: Demonstrate practical, production-oriented AI engineering skills by building a working LangChain agent with real data, real evaluations, and real deployment considerations.

Objectives:

- Build a ReAct-pattern LangChain agent that reasons across multiple financial tools
- Implement at least 6 domain-specific tools wrapping real financial data
- Wire LangSmith for automatic tracing of every agent run
- Design and run structured evaluations (correctness + tool selection) using LangSmith
- Implement conversation memory so multi-turn questions work naturally
- Write a robust data layer that reads from Kaggle CSV with JSON fallback
- Demonstrate Azure deployment readiness via environment-based LLM switching
- Document the full implementation journey including real errors and solutions

What I Am Hoping to Learn

Area	What I want to understand
LangChain agents	How ReAct reasoning loops work in practice — not just theory
LangSmith	How to trace, evaluate, and compare agent runs using LLM-as-judge scoring

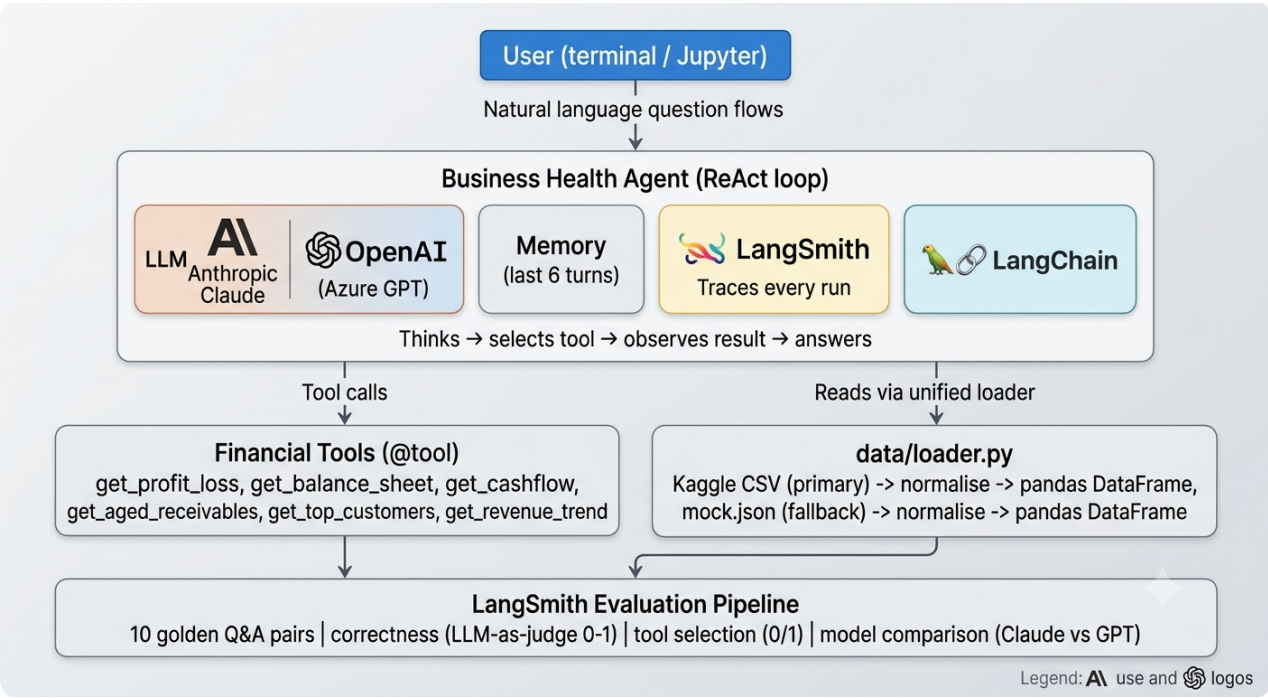
Area	What I want to understand
Prompt engineering	How system prompt structure affects agent reliability and tool selection
Data engineering for AI	How to normalise raw CSV data into a schema that tools can reason over
LLM provider switching	How to build provider-agnostic code (Azure OpenAI / Claude / OpenAI)
Evaluation design	How to write golden Q&A pairs that measure what actually matters
Python packaging	How dependency conflicts in fast-moving ML libraries are diagnosed and resolved
AI security	Why secret scanning, .gitignore, and key rotation matter in AI projects

Technology Stack

Layer	Technology	Purpose
Agent framework	LangChain (langchain-classic)	ReAct agent loop, tool binding, memory
LLM — primary	Anthropic Claude Sonnet 4.5	Agent reasoning and LLM-as-judge evaluation
LLM — Azure path	Azure OpenAI (GPT-4o)	Production LLM via Azure AI Foundry
Tracing & evals	LangSmith	Automatic run logging, eval datasets, scoring
Data processing	pandas, NumPy	CSV ingestion, aggregation, financial calculations
Data source	Kaggle Small Business Financial Dataset 2022–2023	Real transaction-level financial data
Fallback data	Mock JSON (mock.json)	Self-contained demo without Kaggle dependency

Layer	Technology	Purpose
Runtime	Python 3.14, virtualenv	Isolated dependency management
Version control	Git, GitHub	Source control with secret scanning protection
Configuration	python-dotenv	Environment-based secrets management
Notebook	Jupyter	Interactive exploration and demo
Target deployment	Azure Container Apps	Containerised production deployment

Architecture Diagram



Project Structure

```
business-health-agent/  
├── data/  
│   ├── loader.py      ← unified CSV + JSON loader (edit COLUMN_MAP here)  
│   ├── mock.json      ← fallback data (no Kaggle needed)  
│   └── kaggle/         ← drop your Kaggle CSV files here  
│       └── *.csv  
├── tools/  
│   ├── financial_tools.py ← 7 LangChain @tools (CSV-aware)  
│   └── inspect_schema.py  ← run this first to check your CSV columns  
├── agents/  
│   └── business_health_agent.py  
└── evals/
```

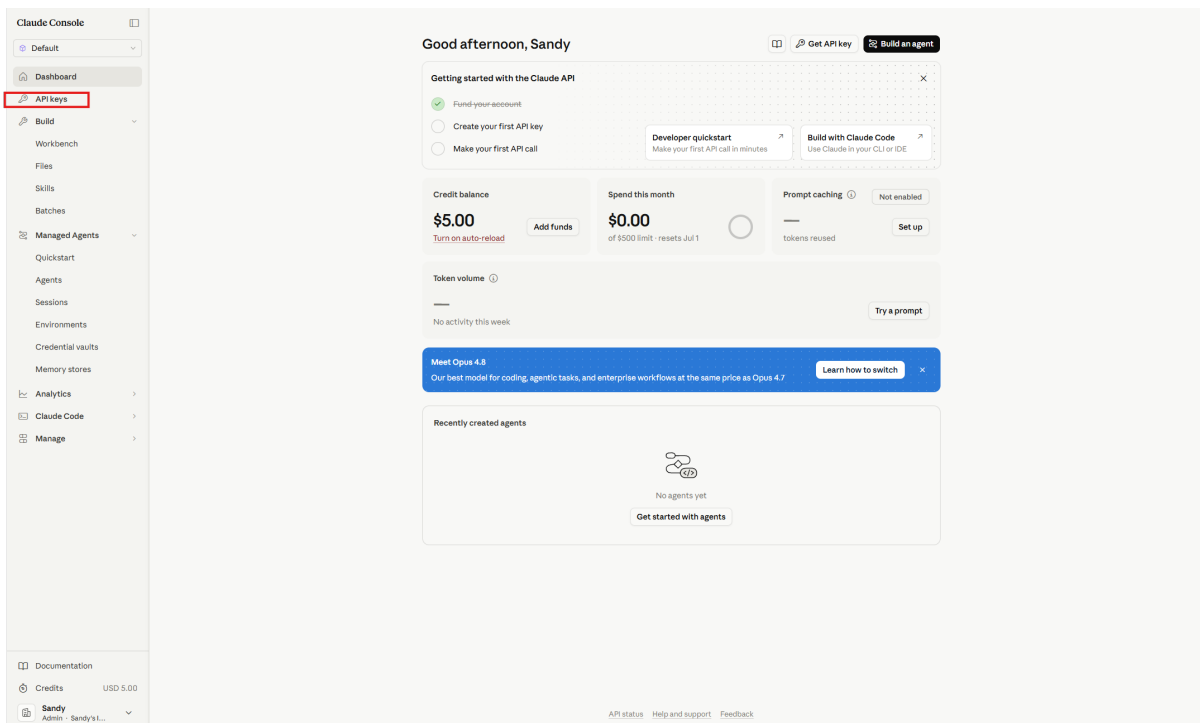
```
├── golden_qa.json
├── run_evals.py
├── compare_models.py
├── config/
├── .env.example
└── requirements.txt
```

Implementation Steps

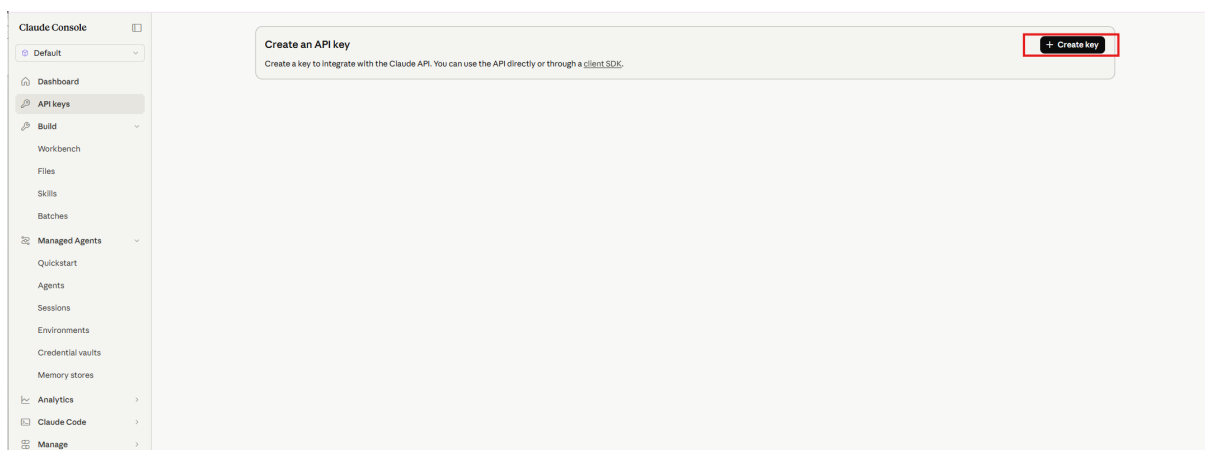
Step 1 : Purchase/Procure API Keys for the Project

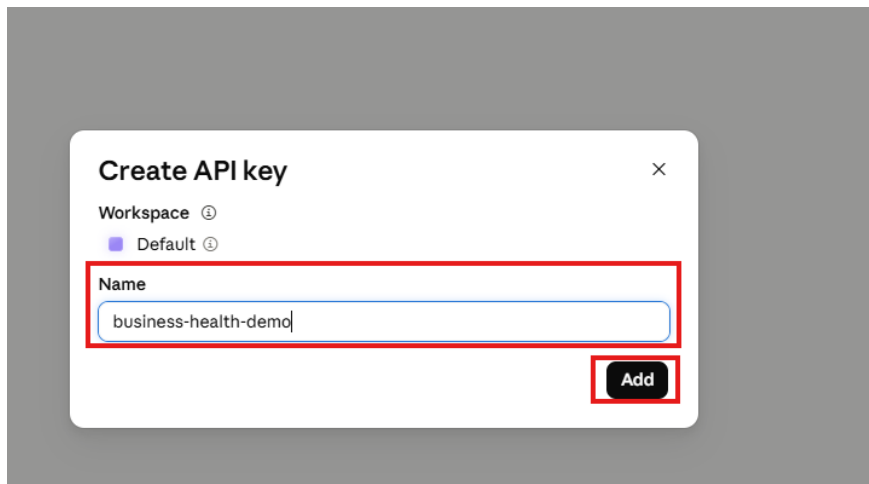
Anthropic API

1. Go to console.anthropic.com
2. Sign up with your email or if you already have Claude Setup
3. Once in, click **API Keys** in the left sidebar

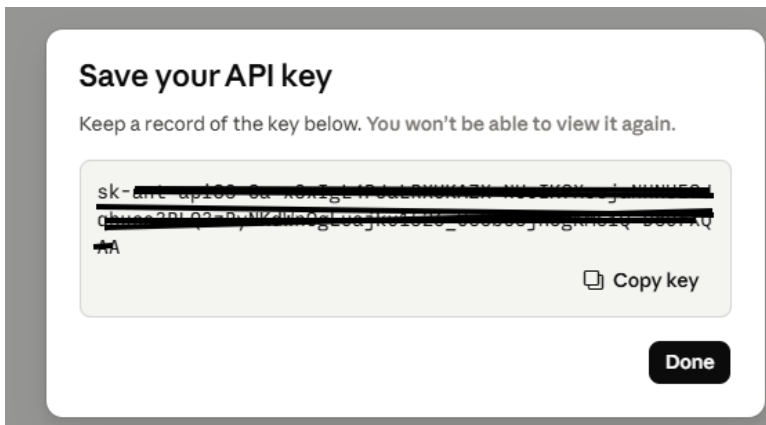


4. Click **Create Key**, name it **business-health-demo**



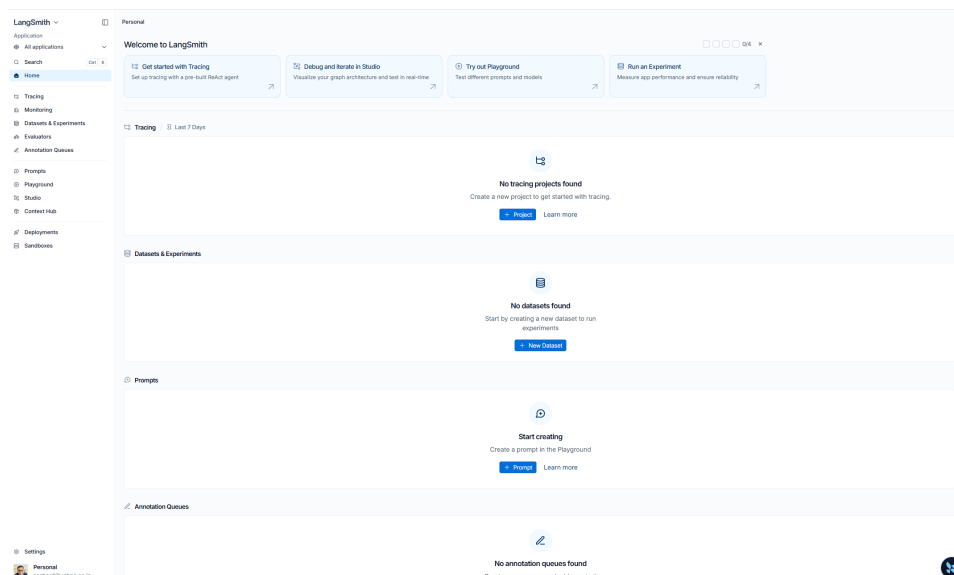


5. Copy the key — it starts with sk-ant-...

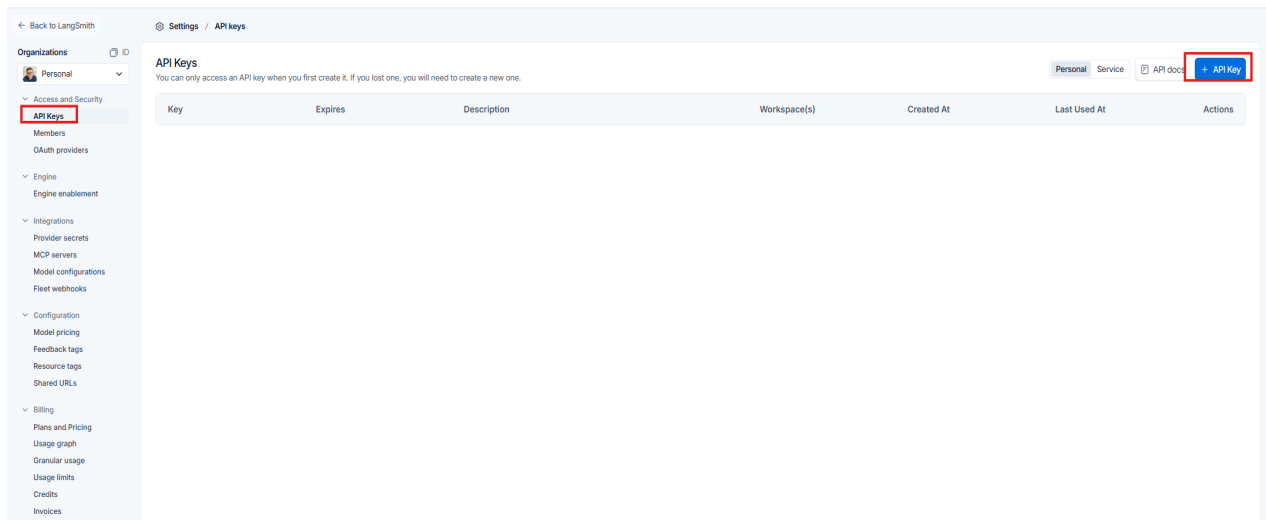


LangSmith API key

1. Go to smith.langchain.com

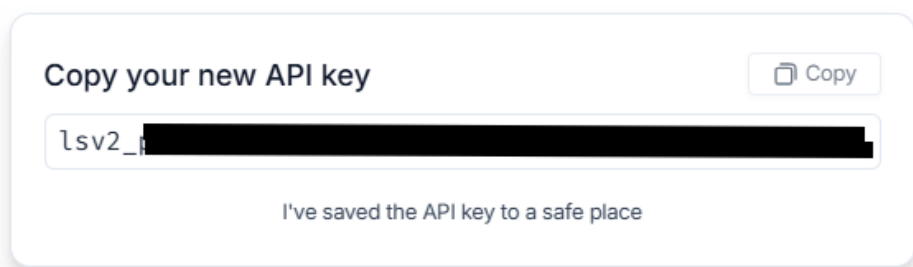


2. Sign up (free, no credit card)
3. Click your avatar → **Settings** → **API Keys**



- Click **Create API Key**, name it **business-health-demo**

- Copy the key — it starts with lsv2_...



Step 2 : Setting up and Install Python in VS Code and activating environments

We now need to move to VS Code and activate the following terminal

Since this is running on Mac OS – we need to validate if we are running zsh or bash

To do this run

`Echo $SHELL`

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS CODE REFERENCE LOG

```
● sandeephegde@Sandeeps-MacBook-Pro-2023 business-health-agent % echo $SHELL  
/bin/zsh
```

Now we need to select the correct Python Interpreter

In VS Code :

1. Press Cmd + Shift + P
2. Type : **Python : Select Interpreter**
3. Choose your Python installation or virtual environment

`python3 -m venv .venv && source .venv/bin/activate`

```
● sandeephegde@Sandeeps-MacBook-Pro-2023 business-health-agent % python3 -m venv .venv && source .venv/bin/activate  
○ (.venv) sandeephegde@Sandeeps-MacBook-Pro-2023 business-health-agent %
```

We will now run the following command below :

`pip install -r requirements.txt`

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS CODE REFERENCE LOG

```
● (.venv) sandeephegde@Sandeeps-MacBook-Pro-2023 business-health-agent % pip install -r requirements.txt  
Requirement already satisfied: langchain>=0.3.0 in ./venv/lib/python3.14/site-packages (from -r requirements.txt (line 5)) (1.3.4)  
Requirement already satisfied: langchain-core>=0.3.0 in ./venv/lib/python3.14/site-packages (from -r requirements.txt (line 6)) (1.4.1)  
Requirement already satisfied: langchain-community>=0.3.0 in ./venv/lib/python3.14/site-packages (from -r requirements.txt (line 7)) (0.4.2)  
Requirement already satisfied: langchain-openai>=0.2.0 in ./venv/lib/python3.14/site-packages (from -r requirements.txt (line 10)) (1.2.2)  
Requirement already satisfied: langchain-anthropic>=0.3.0 in ./venv/lib/python3.14/site-packages (from -r requirements.txt (line 11)) (1.4.4)  
Requirement already satisfied: pandas>=2.0.0 in ./venv/lib/python3.14/site-packages (from -r requirements.txt (line 14)) (3.0.3)  
Requirement already satisfied: numpy>=1.24.0 in ./venv/lib/python3.14/site-packages (from -r requirements.txt (line 15)) (2.4.6)  
Requirement already satisfied: langsmith>=0.2.0 in ./venv/lib/python3.14/site-packages (from -r requirements.txt (line 18)) (0.8.9)  
Requirement already satisfied: python-dotenv>=1.0.0 in ./venv/lib/python3.14/site-packages (from -r requirements.txt (line 21)) (1.2.2)  
Requirement already satisfied: kaggle>=1.6.0 in ./venv/lib/python3.14/site-packages (from -r requirements.txt (line 22)) (2.2.1)  
Requirement already satisfied: langgraph<1.3.0,>=1.2.4 in ./venv/lib/python3.14/site-packages (from langchain>=0.3.0->-r requirements.txt (line 5)) (1.2.4)  
Requirement already satisfied: pydantic<3.0.0,>=2.7.4 in ./venv/lib/python3.14/site-packages (from langchain>=0.3.0->-r requirements.txt (line 5)) (2.13.4)  
Requirement already satisfied: jsonpatch<2.0.0,>=1.33.0 in ./venv/lib/python3.14/site-packages (from langchain-core>=0.3.0->-r requirements.txt (line 6)) (1.33)  
Requirement already satisfied: langchain-protocol>=0.0.14 in ./venv/lib/python3.14/site-packages (from langchain-core>=0.3.0->-r requirements.txt (line 6)) (0.0.16)  
Requirement already satisfied: packaging>=23.2.0 in ./venv/lib/python3.14/site-packages (from langchain-core>=0.3.0->-r requirements.txt (line 6)) (26.2)  
Requirement already satisfied: pyyaml<7.0.0,>=5.3.0 in ./venv/lib/python3.14/site-packages (from langchain-core>=0.3.0->-r requirements.txt (line 6)) (6.0.3)  
Requirement already satisfied: tenacity!=8.4.0,<10.0.0,>=8.1.0 in ./venv/lib/python3.14/site-packages (from langchain-core>=0.3.0->-r requirements.txt (line 6)) (9.1.4)  
Requirement already satisfied: typing-extensions<5.0.0,>=4.7.0 in ./venv/lib/python3.14/site-packages (from langchain-core>=0.3.0->-r requirements.txt (line 6)) (4.15.0)  
Requirement already satisfied: uuid-utils<1.0,>=0.12.0 in ./venv/lib/python3.14/site-packages (from langchain-core>=0.3.0->-r requirements.txt (line 6)) (0.16.0)  
Requirement already satisfied: httpx<1,>=0.23.0 in ./venv/lib/python3.14/site-packages (from langsmith>=0.2.0->-r requirements.txt (line 18)) (0.28.1)
```

Step 3 : Configure the API Keys in the config ./env.example file

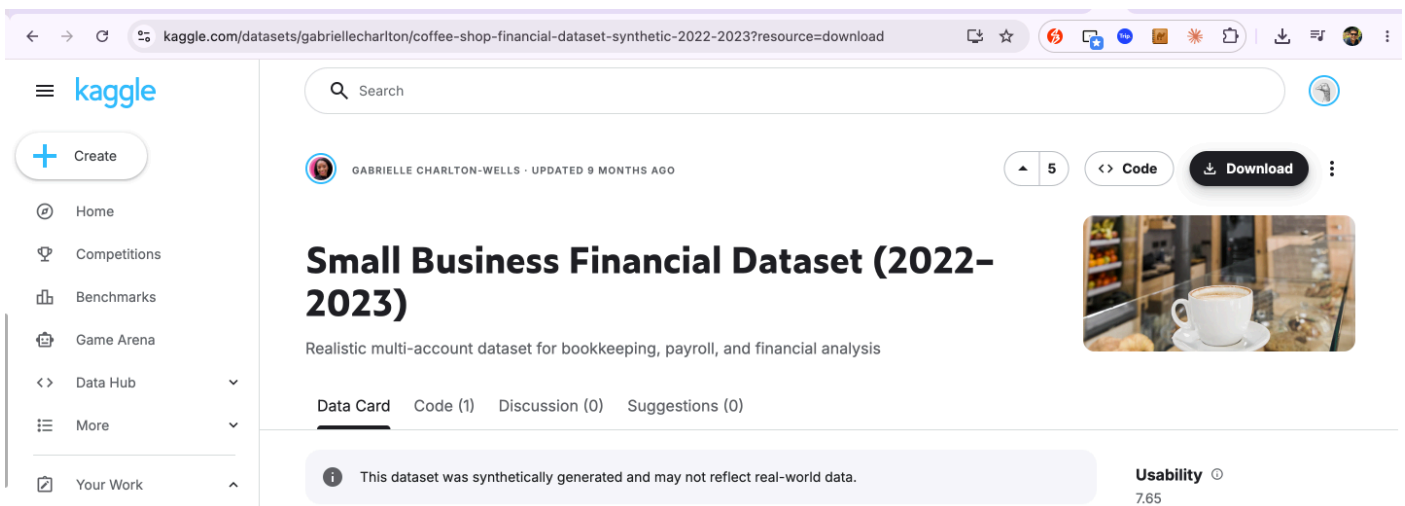
We will now move to the Configure option and we will enter the API keys from Step 1 in the file below

`cp config/.env.example config/.env`

```
config > $ .env.example
1 # Business Health Agent - Environment Variables
2 # Copy to config/.env and fill in your values
3
4 # -- LangSmith (tracing + evals) -----
5 LANGSMITH_API_KEY=lsv2_pt 1
6 LANGSMITH_TRACING=true
7 LANGSMITH_PROJECT=business-health-agent
8
9 # -- LLM -- choose ONE -----
10
11 # Option A: Anthropic Claude (recommended - easiest to get started)
12 ANTHROPIC_API_KEY=sk-ant-api0:
13 ANTHROPIC_MODEL=claude-sonnet-4-20250514
14
15 # Option B: Azure OpenAI (uncomment + comment out Option A above)
16 # AZURE_OPENAI_API_KEY=your_azure_key
17 # AZURE_OPENAI_ENDPOINT=https://your-resource.openai.azure.com/
18 # AZURE_OPENAI_DEPLOYMENT=gpt-4o
19 # AZURE_OPENAI_API_VERSION=2024-02-01
20
21 # Option C: OpenAI direct (uncomment + comment out Option A above)
22 # OPENAI_API_KEY=your_openai_key
23 # OPENAI_MODEL=gpt-4o
24
```

Step 4 : Download the *.csv files from Kaggle.com and add them in the folder

<https://www.kaggle.com/datasets/gabriellecharlton/coffee-shop-financial-dataset-synthetic-2022-2023?resource=download>



Step 5 : Run the agent

Interactive

```
python agents/business_health_agent.py
```

Single question

```
python agents/business_health_agent.py -q "Is the business profitable this month?"
```

Streaming

```
python agents/business_health_agent.py --stream
```


Results

python agents/business_health_agent.py

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS CODE REFERENCE LOG

```
- **August 2025:** Declined 4.3% to $29,015 (lowest point)
- **September 2025:** Rebounded strongly, up 10.0% to $31,913
- **October 2025:** Continued growth, up 10.3% to $35,193 (peak month)
- **November 2025:** Slight dip of 3.2% to $34,060
- **December 2025:** Essentially flat at $34,053

**Key Takeaways:**
- The business recovered well from the August low point
- Revenue has stabilized around $34,000-$35,000 in recent months
- While there's month-to-month volatility, the overall trajectory is positive
- The business is currently performing about 12% above where it started six months ago

The recent flatness (November-December) is worth monitoring to ensure the growth momentum continues into the new year.
/Users/sandeephegde/Desktop/business-health-agent/.venv/lib/python3.14/site-packages/langchain_classic/chains/base.py:491:
UserWarning: 'ConversationBufferWindowMemory' got multiple output keys: dict_keys(['output', 'intermediate_steps']). The
default 'output' key is being used. If this is not desired, please manually set 'output_key'.
  self.memory.save_context(inputs, outputs)

> Finished chain.

 Answer:
**Revenue is growing** over the past six months, though with some fluctuations.

Here are the key details:

**Overall Trend:** Growing (up 12.3% from July to December 2025)
- July 2025: $30,318
- December 2025: $34,053
- Average monthly revenue: $32,425

**Monthly Performance:**
- **August 2025:** Declined 4.3% to $29,015 (lowest point)
- **September 2025:** Rebounded strongly, up 10.0% to $31,913
- **October 2025:** Continued growth, up 10.3% to $35,193 (peak month)
- **November 2025:** Slight dip of 3.2% to $34,060
- **December 2025:** Essentially flat at $34,053

**Key Takeaways:**
- The business recovered well from the August low point
- Revenue has stabilized around $34,000-$35,000 in recent months
- While there's month-to-month volatility, the overall trajectory is positive
- The business is currently performing about 12% above where it started six months ago

The recent flatness (November-December) is worth monitoring to ensure the growth momentum continues into the new year.

❖ (.venv) sandeephegde@Sandeeps-MacBook-Pro-2023 business-health-agent %
```

- (.venv) sandeephegde@sandeeps-MacBook-Pro-2023 business-health-agent % python agents/business_health_agent.py -q "how much did the business spend in wages?"

Business Health Agent

LLM: Claude Sonnet | LangSmith: ✓ active
Type 'exit' to quit

```
/Users/sandeephegde/Desktop/business-health-agent/agents/business_health_agent.py:105: LangChainDeprecationWarning: The class `ConversationBufferWindowMemory` was deprecated in LangChain 0.3.1 and will be removed in 2.0.0. Use `langchain.agents.create_agent` instead. For agents that need to remember prior interactions, use `create_agent` with checkpointing or the `Store` API. See https://docs.langchain.com/oss/python/langchain/short-term-memory and https://docs.langchain.com/oss/python/langchain/long-term-memory
memory = ConversationBufferWindowMemory(
```

> Entering new AgentExecutor chain...

I need to find out how much the business spent on wages. Wages are typically recorded as an operating expense in the profit and loss statement. I should get the P&L data to see the wages expense. Let me start with the most recent month.

Action: get_profit_loss

Action Input: this-month

```
✓ Loaded checking_account_main.csv (877 rows, columns: ['date', 'transaction_id', 'description', 'category', 'type', 'amount', 'balance', '_source_file'])
✓ Loaded checking_account_secondary.csv (48 rows, columns: ['date', 'transaction_id', 'description', 'category', 'type', 'amount', 'balance', '_source_file'])
✓ Loaded credit_card_account.csv (133 rows, columns: ['date', 'transaction_id', 'vendor', 'category', 'type', 'amount', 'balance', '_source_file'])
✓ Loaded gusto_payroll.csv (92 rows, columns: ['pay_date', 'employee_id', 'employee_name', 'role', 'type', 'amount', 'account', '_source_file'])
```

 Data source: Kaggle CSV (1,058 rows total)

```
{'period': '2025-12', 'data_rows_analysed': 47, 'revenue': {'breakdown': {'Daily Sales Deposit': 29045.0, 'Transfer from Main': 5008.0}, 'total': np.float64(34053.0)}, 'cost_of_goods_sold': np.float64(1421.0), 'gross_profit': np.float64(32632.0), 'gross_margin_pct': np.float64(95.8), 'operating_expenses': {'breakdown': {'Payroll Funding': 4725.0, 'Utility Bill Payment': 746.0, 'Rent Payment': 287.0, 'Maintenance Payment': 218.0}, 'total': np.float64(7531.0)}, 'operating_profit': np.float64(25101.0), 'estimated_tax': np.float64(7530.3), 'net_profit': np.float64(17570.7), 'net_margin_pct': np.float64(51.6), 'is_profitable': np.True_} I can see wages information for this month, but to give a complete picture of total wage spending, I should check if there's data for the full year or at least a few more months to provide better context.
```

Action: get_profit_loss

Action Input: 2025

```
{'period': '2025-12', 'data_rows_analysed': 47, 'revenue': {'breakdown': {'Daily Sales Deposit': 29045.0, 'Transfer from Main': 5008.0}, 'total': np.float64(34053.0)}, 'cost_of_goods_sold': np.float64(1421.0), 'gross_profit': np.float64(32632.0), 'gross_margin_pct': np.float64(95.8), 'operating_expenses': {'breakdown': {'Payroll Funding': 4725.0, 'Utility Bill Payment': 746.0, 'Rent Payment': 287.0, 'Maintenance Payment': 218.0}, 'total': np.float64(7531.0)}, 'operating_profit': np.float64(25101.0), 'estimated_tax': np.float64(7530.3), 'net_profit': np.float64(17570.7), 'net_margin_pct': np.float64(51.6), 'is_profitable': np.True_} The year 2025 data appears to only show December. Let me check previous months to
```

```
at64(32260.0), 'gross_margin_pct': np.float64(94.7), 'operating_expenses': {'breakdown': {'Payroll Funding': 3073.0, 'Rent Payment': 999.0, '': 806.0, 'Maintenance Payment': 244.0}}, 'total': np.float64(5122.0)}, 'operating_profit': np.float64(27138.0), 'estimated_tax': np.float64(8141.4), 'net_profit': np.float64(18996.6), 'net_margin_pct': np.float64(55.8), 'is_profitable': np.True_} I can see wage data for the last two months. Let me check one more month to provide a better trend.
```

Action: get_profit_loss

Action Input: two-months-ago

```
{'period': '2025-10', 'data_rows_analysed': 45, 'revenue': {'breakdown': {'Daily Sales Deposit': 29226.0, 'Transfer from Main': 4646.0, '': 1321.0}}, 'total': np.float64(35193.0)}, 'cost_of_goods_sold': np.float64(667.0), 'gross_profit': np.float64(34526.0), 'gross_margin_pct': np.float64(98.1), 'operating_expenses': {'breakdown': {'Payroll Funding': 4433.0, '': 598.0, 'Maintenance Payment': 516.0, 'Utility Bill Payment': 402.0}}, 'total': np.float64(5949.0)}, 'operating_profit': np.float64(28577.0), 'estimated_tax': np.float64(8573.1), 'net_profit': np.float64(20003.9), 'net_margin_pct': np.float64(56.8), 'is_profitable': np.True_} I now have wage data for the last three months. I can see "Payroll Funding" in the operating expenses, which represents wages. Let me provide a comprehensive answer.
```

Final Answer: Based on the profit and loss data, the business spent on wages (recorded as "Payroll Funding") as follows:

****Recent Wage Spending:****

- ****December 2025 (this month):**** \$4,725.00

- ****November 2025:**** \$3,073.00

- ****October 2025:**** \$4,433.00

****3-Month Total:**** \$12,231.00

****Average Monthly Wages:**** \$4,077.00

The wage expense shows some variation month-to-month, ranging from approximately \$3,073 to \$4,725. December had the highest payroll cost at \$4,725, which represents about 14% of total revenue for that month. Overall, wages are a significant but manageable operating expense, accounting for roughly 12-14% of monthly revenue across these three months.

UserWarning: 'ConversationBufferWindowMemory' got multiple output keys: dict_keys(['output', 'intermediate_steps']). The default 'output' key is being used. If this is not desired, please manually set 'output_key'.
self.memory.save_context(inputs, outputs)

> Finished chain.

 Answer:

Based on the profit and loss data, the business spent on wages (recorded as "Payroll Funding") as follows:

****Recent Wage Spending:****

- ****December 2025 (this month):**** \$4,725.00

- ****November 2025:**** \$3,073.00

- ****October 2025:**** \$4,433.00

****3-Month Total:**** \$12,231.00

****Average Monthly Wages:**** \$4,077.00

The wage expense shows some variation month-to-month, ranging from approximately \$3,073 to \$4,725. December had the highest payroll cost at \$4,725, which represents about 14% of total revenue for that month. Overall, wages are a significant but manageable operating expense, accounting for roughly 12-14% of monthly revenue across these three months.

Langsmith Error Tracing

Trace

AgentExecutor

claude-sonnet-4-5

3.53s 1K

get_profit_loss 0.05s

claude-sonnet-4-5

3.62s 1.3K

get_profit_loss 0.00s

claude-sonnet-4-5

2.04s 1.7K

get_profit_loss 0.01s

claude-sonnet-4-5

2.06s 2K

get_profit_loss 0.01s

claude-sonnet-4-5

6.10s 2.5K

ReActSingleInputOutputParser 0.00s

ReActSingleInputOutputParser

Feedback Input Output Attributes

Feedback

+ Add feedback

Input

AI

I now have wage data for the last three months. I can see "Payroll Funding" in the operating expenses, which represents wages. Let me provide a comprehensive answer.
Final Answer: Based on the profit and loss data, the business spent on wages (recorded as "Payroll Funding") as follows:
Recent Wage Spending:

- December 2025 (this month): \$4,725.00
- November 2025: \$3,073.00
- October 2025: \$4,433.00

3-Month Total: \$12,231.00
Average Monthly Wages: \$4,077.00
The wage expense shows some variation month-to-month, ranging from approximately \$3,073 to \$4,725. December had the highest payroll cost at \$4,725, which represents about 14% of total revenue for that month. Overall, wages are a significant but manageable operating expense, accounting for roughly 12-14% of monthly revenue across these three months.

Output

platform.claude.com/settings/workspaces/default/keys

Claude Console

Default

Dashboard

API keys

Build

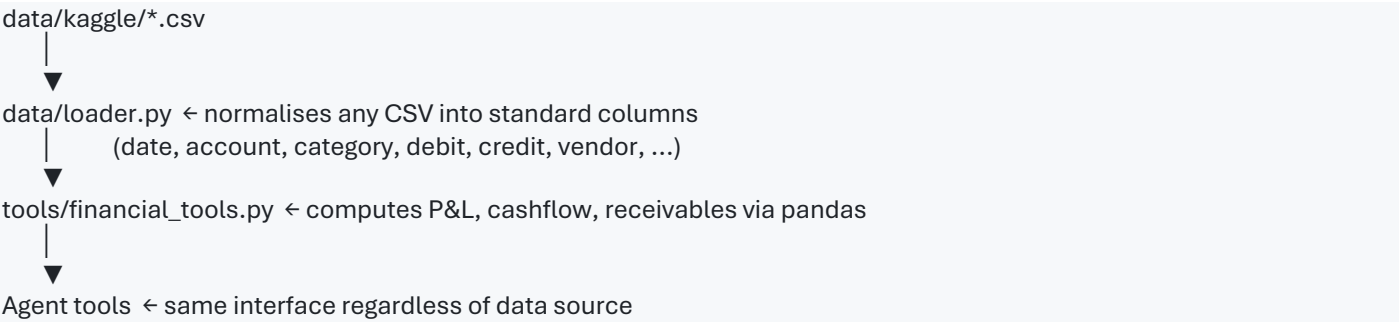
API keys 2

API keys are owned by workspaces and remain active even after the creator is removed

Key	Created by	Created at	Last used at	Cost
business-health-demo sk-ant-api03-4eh...HwAA	Sandy sashcell@yahoo.co.in	Jun 6, 2026	Jun 6, 2026	USD 0.21

+ Create key

How CSV support works



If your CSV has different column names, edit COLUMN_MAP in data/loader.py — no other files need changing.

Tools

Tool	What it computes
<code>get_profit_loss(period)</code>	Revenue, COGS, opex, net profit, margins
<code>get_balance_sheet(date)</code>	Assets, liabilities, equity, ratios
<code>get_cashflow(period)</code>	Operating/investing/financing flows
<code>get_cashflow_forecast(months)</code>	Trend-based projection + risk flag
<code>get_aged_receivables()</code>	Outstanding by vendor, aging buckets
<code>get_top_customers(limit)</code>	Revenue by customer/vendor
<code>get_revenue_trend(months)</code>	Month-by-month growth + direction

Challenges and Solutions

1. LangChain breaking changes — *AgentExecutor* import failure

Challenge: After installing the latest LangChain (1.x), `from langchain.agents import AgentExecutor` raised an `ImportError`. The class had moved entirely.

Solution: Traced the new package structure and found `AgentExecutor` had been moved to `langchain-classic`, a compatibility shim package. Updated all affected imports (`agents`, `memory`, `prompts`) to reference `langchain_classic`. Added explicit version pinning guidance to `requirements.txt`.

Learning: Fast-moving ML libraries break imports between minor versions. Always test installs in a clean `virtualenv` and check the migration guide before upgrading.

2. CSV data source integration — *schema uncertainty*

Challenge: The Kaggle dataset is login-gated, so the exact column names couldn't be verified without downloading it. Writing tools against assumed column names would break silently.

Solution: Built a flexible `COLUMN_MAP` dictionary in `data/loader.py` that maps many possible column name variants to internal standard names. Added `tools/inspect_schema.py` — a diagnostic script that prints every CSV column, its data type, sample values, and which `COLUMN_MAP` entries matched or failed. The agent works with mock JSON if no CSV is present, giving a working demo regardless.

Learning: Defensive data loading with explicit column mapping is more robust than assuming schema. A diagnostic script pays for itself in the first debugging session.

3. LLM model name mismatch — 404 from Anthropic API

Challenge: The agent started successfully but crashed with a 404 error: model: claude-sonnet-4-20250514 not found. The model string in .env was wrong for the API tier.

Solution: Updated ANTHROPIC_MODEL in config/.env to claude-sonnet-4-5, which is the correct model identifier for the available API access level.

Learning: Model identifiers are version-specific and tier-specific. Always verify the exact string against the provider's current model list, not documentation that may be outdated.

4. Balance sheet computed from transactions — approximation risk

Challenge: The Kaggle dataset contains raw transactions, not pre-computed financial statements. A real balance sheet requires an accounting system; deriving one from transactions requires assumptions.

Solution: Built the get_balance_sheet tool to compute approximate figures from transaction history (cash = net of all credits/debits, receivables = 15% of credited revenue, inventory = 20% of COGS). Added an explicit note field in the tool output stating the figures are computed approximations. The mock JSON fallback contains hand-crafted realistic balance sheet figures for demo purposes.

Learning: Transparency about data quality is as important as the data itself. An agent that presents approximations as facts is more dangerous than one that clearly labels its assumptions.

Key Learnings

On AI engineering in practice: Building a working agent is 20% prompt design and 80% data plumbing, dependency management, and error handling. The theoretical concepts (ReAct, tool use, memory) are straightforward — the engineering discipline to make them reliable is the real skill.

On LangSmith: Tracing is not optional for agent debugging. Without LangSmith, a wrong answer from the agent is a black box. With tracing, you can see exactly which tool was called, what it returned, how the LLM interpreted it, and where the reasoning went wrong. Evals with LLM-as-judge scoring provide a reproducible benchmark that lets you compare models or prompt changes objectively.

On LangChain: The library is powerful but evolves rapidly. Understanding the underlying pattern (prompt → LLM → parser → tool loop) matters more than memorising the current import paths, which change between versions.

On security: API key hygiene is not bureaucracy — it is a core engineering responsibility. Secrets in git history are permanently compromised even after deletion, because the history is replicated across every clone. Rotate first, fix second.

On Azure readiness: Environment-based LLM switching (Azure OpenAI / Claude / OpenAI via a single env var) is the right pattern for enterprise deployment. It means the same codebase runs locally with Claude and in production with Azure OpenAI without any code changes.